

UNIT – 3
ASSESSMENT – 1
SOURCECODE

NIHARIKA PREEJESH 192524506

1. # Smart Medical Diagnosis System
Forward Chaining and Backward Chaining

```
facts = {  
    "fever",  
    "cough",  
    "breathlessness"  
}
```

```
rules = [  
    ({ "fever", "cough" }, "flu"),  
    ({ "fever", "rash" }, "measles"),  
    ({ "cough", "breathlessness" }, "pneumonia")  
]
```

FORWARD CHAINING

```
# -----
```

```
def forward_chaining(facts, rules):
```

```
    known = set(facts)
```

```
    print("\n===== FORWARD CHAINING =====")
```

```
    changed = True
```

```
    while changed:
```

```
        changed = False
```

```
        for conditions, conclusion in rules:
```

```
            if conditions.issubset(known):
```

```
                if conclusion not in known:
```

```
                    print("\nRule Fired:")
```

```
                    print(
```

```
        " AND ".join(conditions),  
        "->",  
        conclusion  
    )
```

```
    known.add(conclusion)
```

```
    print("Derived Fact:", conclusion)
```

```
    changed = True
```

```
    return known
```

```
# -----
```

```
# BACKWARD CHAINING
```

```
# -----
```

```
def backward_chaining(goal, facts, rules, visited=None):
```

```
    if visited is None:
```

```
        visited = set()
```

```
print("\nChecking Goal:", goal)
```

```
if goal in facts:
```

```
    print(goal, "is already a fact.")
```

```
    return True
```

```
if goal in visited:
```

```
    return False
```

```
visited.add(goal)
```

```
for conditions, conclusion in rules:
```

```
    if conclusion == goal:
```

```
        print(
```

```
            "Using Rule:",
```

```
            " AND ".join(conditions),
```

```
            "->",
```

```
            conclusion
```

```
        )
```

```
all_true = True
```

```
for condition in conditions:
```

```
    if not backward_chaining(
```

```
        condition,
```

```
        facts,
```

```
        rules,
```

```
        visited
```

```
    ):
```

```
        all_true = False
```

```
if all_true:
```

```
    return True
```

```
return False
```

```
# -----
```

```
# MAIN PROGRAM
```

```
# -----
```

```
print("SMART MEDICAL DIAGNOSIS SYSTEM")
```

```
print("\nInitial Facts:")
```

```
for fact in facts:
```

```
    print("-", fact)
```

```
derived = forward_chaining(facts, rules)
```

```
print("\nAll Derived Facts:")
```

```
for fact in derived:
```

```
    print("-", fact)
```

```
goal = "pneumonia"
```

```
print("\n===== BACKWARD CHAINING =====")
```

```
if backward_chaining(goal, facts, rules):
```

```
    print("\nConclusion:")
```

```
    print("Patient A has Pneumonia.")
```

```
else:
```

```
    print("\nConclusion:")
```

```
print("Pneumonia cannot be proved.")
```

```
2. # Autonomous Traffic Management Agent
```

```
# FOL reasoning using substitution and resolution-style  
inference
```

```
facts = {  
    "Vehicle(Ambulance)",  
    "EmergencyType(Ambulance)",  
    "Vehicle(CarA)",  
    "Behind(CarA,Ambulance)"  
}
```

```
print("==== AUTONOMOUS TRAFFIC MANAGEMENT  
====")
```

```
print("\nInitial Facts:")
```

```
for fact in facts:
```

```
    print(fact)
```

Rule 1

```
print("\n--- Rule 1 ---")
```

```
print("Vehicle(x) AND EmergencyType(x) -> ClearPath(x)")
```

```
if ("Vehicle(Ambulance)" in facts and
```

```
    "EmergencyType(Ambulance)" in facts):
```

```
    print("\nUnification:")
```

```
    print("x = Ambulance")
```

```
    print("MGU = {x/Ambulance}")
```

```
    facts.add("ClearPath(Ambulance)")
```

```
    print("Derived: ClearPath(Ambulance)")
```

Rule 2

```
print("\n--- Rule 2 ---")
```

```
print("ClearPath(x) -> GreenSignal(x)")
```

```
if "ClearPath(Ambulance)" in facts:
```



```
facts.add("GreenSignal(Ambulance)")
```

```
print("Derived: GreenSignal(Ambulance)")
```

```
# Rule 3
```

```
print("\n--- Rule 3 ---")
```

```
print(  
    "Vehicle(x) AND Behind(x,y) AND "  
    "GreenSignal(y) -> Proceed(x)"  
)
```

```
if (  
    "Vehicle(CarA)" in facts and  
    "Behind(CarA,Ambulance)" in facts and  
    "GreenSignal(Ambulance)" in facts  
):
```

```
facts.add("Proceed(CarA)")
```

```
print("Derived: Proceed(CarA)")
```

Resolution

```
print("\n===== RESOLUTION =====")
```

```
print("Goal: Proceed(CarA)")
```

```
print("Negated Goal: NOT Proceed(CarA)")
```

if "Proceed(CarA)" in facts:

```
    print("\nResolution:")
```

```
    print("Proceed(CarA)")
```

```
    print("NOT Proceed(CarA)")
```

```
    print("-----")
```

```
    print("EMPTY CLAUSE ( $\square$ )")
```

```
    print("\nConclusion:")
```

```
    print("Proceed(CarA) is PROVED.")
```

else:

```
    print("Goal cannot be proved.")
```

3. # Agricultural Expert Reasoning System

Resolution Refutation

```
print("=====
```

```
====")

print(" AGRICULTURAL EXPERT REASONING
SYSTEM")
```

```
print("=====
====")
```

Initial facts

```
facts = {
    "SoilDry",
    "CropWheat"
}
```

```
print("\nInitial Facts:")
```

for fact in facts:

```
    print(fact)
```

Rule 1

print("\nRule 1:")

print("SoilDry -> IrrigationNeeded")

if "SoilDry" in facts:

facts.add("IrrigationNeeded")

print("Derived: IrrigationNeeded")

Rule 2

print("\nRule 2:")

print(
 "IrrigationNeeded AND CropWheat "
 "-> ApplyDripMethod"
)

if (
 "IrrigationNeeded" in facts and
 "CropWheat" in facts
):

```
facts.add("ApplyDripMethod")
```

```
print("Derived: ApplyDripMethod")
```

```
# Goal
```

```
goal = "ApplyDripMethod"
```

```
print("\nGoal:", goal)
```

```
print("Negated Goal: NOT ApplyDripMethod")
```

```
# Resolution
```

```
print("\n===== RESOLUTION REFUTATION =====")
```

```
if "ApplyDripMethod" in facts:
```

```
    print("\nDerived Clause:")
```

```
    print("ApplyDripMethod")
```

```
print("\nNegated Goal:")
print("NOT ApplyDripMethod")
```

```
print("\nResolving:")
print("ApplyDripMethod")
print("NOT ApplyDripMethod")
print("-----")
print("EMPTY CLAUSE (□)")
```

```
print("\nConclusion:")
print("ApplyDripMethod is PROVED.")
```

else:

```
print("\nApplyDripMethod cannot be proved.")
```

Crop At Risk Analysis

```
print("\n===== CROP RISK ANALYSIS =====")
```

if (

"CropWheat" in facts and

```
"NOT ApplyDripMethod" in facts  
):
```

```
print("CropAtRisk is TRUE.")
```

```
else:
```

```
print(  
    "CropAtRisk cannot be concluded "  
    "from the given facts."  
)
```